A Deep Dive into Deep Knowledge Tracing

Botond Kovács

1 Introduction

In my data analysis project, I focused on the concept of personalized education, specifically, on the Deep Knowledge Tracing paper[1], which introduced a neural network-based approach to modeling student learning.

My initial goals were to implement the models from the paper, modify them to improve AUC, and extract the latent structure in the data using a novel approach.

2 Data Cleaning

The aforementioned paper and my project focused on a widely used dataset in personalized education, the ASSISTments 2009-2010 dataset[2]. It consists of online learning sequences, that track the progress of students' as they answer different questions. To each student a sequence belongs, with each data point representing a question attempt. The key attributes are the timestamp of the attempt, the question ID, whether the student answered correctly (correctness), and the associated skill ID.

Regarding data cleaning, there are two aspects that differ significantly from the paper.

2.1 Data Filtering

Firstly, I implemented a more sophisticated data filtering process. Instead of only dropping students who only answered one question, I excluded questions belonging to underrepresented skills - those with not enough data for a meaningful analysis. For further details, see the code in my project's GitHub repository[3].

Version	#Users	#Skills	#Answers
Original	4 217	124	525 534
Filtered	3 657	109	301 088

Table 1: Data before and after filtering

2.2 Multi-Skill Representation

Secondly, I identified a potential flaw in the way the original paper handled the data. In the dataset, each entry is associated with both a question ID and a skill ID, where the skill ID represents a broader category (e.g., Statistics or Linear Algebra). However, some questions belong to multiple skills.

When this occurs, the dataset duplicates the corresponding entry, differing only in the skill ID column. In the original implementation[1] these duplicates were treated as separate entries within the same sequence, which could artificially inflate model performance.

To address this, I restructured the data by creating binary columns — one for each skill ID — and merging duplicated entries into a single row. This makes sure that multi-skill questions are properly represented (see Figure 1).

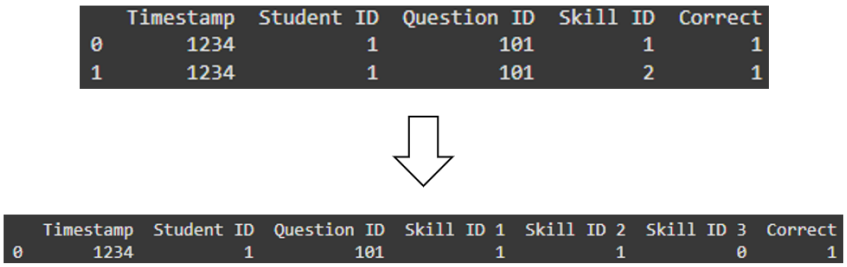

Figure 1: An example of the skill ID transformation.

As for the RNN-based models the skills are transformed into an embedding, this change required me to come up with a custom embedding layer. This layer works as expected[4], when a question only belongs to one skill, but in case of multiple skills, its embedding is the mean of the individual embeddings of the corresponding skills.

3 Bayesian Knowledge Tracing (BKT)

After thorough data pre-processing I familiarized myself with a fundamental approach to model student learning: Bayesian Knowledge Tracing[5]. This method uses Hidden Markov models to represent a student's knowledge state as the latent variable and their answers as observations (see Figure 2).

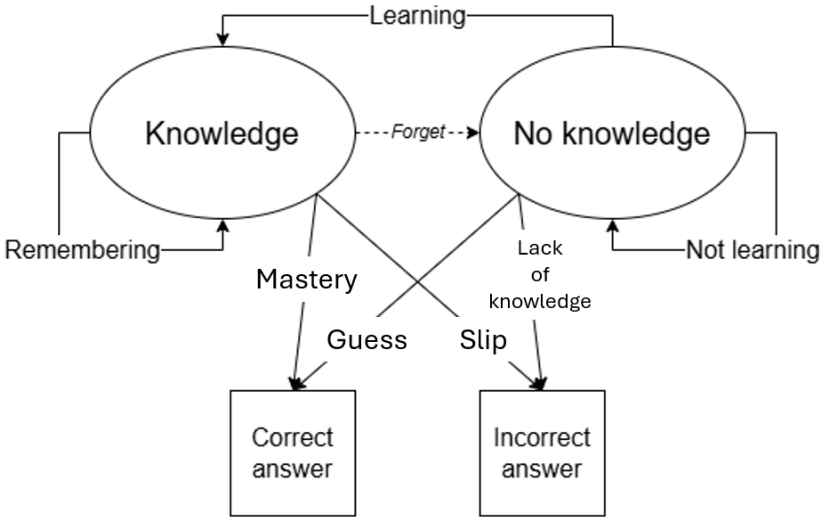

Figure 2: Basci model of learning a skill

The biggest flaw of this approach is the assumption that skills are independent of each other, requiring a separate model for each skill.

My contributions in this section are implementing the method, achieving similar performance, and exploring the effects of allowing to forget, contrary to what they assumed in the original paper.

Allowing forget simply means permitting a nonzero probability of going back to not knowing a skill from knowing a skill.

During modeling I split the dataset into two main sections: **80% training** set and **20% test** set. With my implementation, I was able to replicate the **0.67 average AUC** on the test set. After allowing to forget, the test AUC increased to **0.70** (see Figure 3).

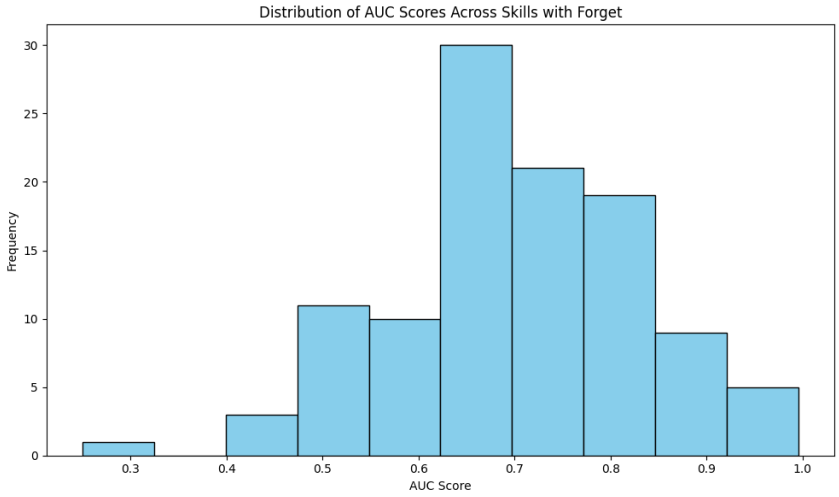

Figure 3

4 Deep Knowledge Tracing (DKT)

As stated in the original paper, all of the flaws of BKT can be tackled by using recurrent neural networks for student learning, which is called Deep Knowledge Tracing.

It allows us to use a single model for various skills while incorporating more complex connections.

4.1 Inputs and Outputs

The neural network's input is a sequence of answers, where each answer is paired with its corresponding skill ID, correctness, and any additional available features.

The output is a vector for each item in the sequence—let's call it a knowledge vector. This vector has a size equal to the number of skills in the data, with each coordinate representing the probability that the user would correctly answer a question for a given skill in that state.

Example Outputs

Figure 4 illustrates a simple example. Imagine a person learning three mathematical skills with no prior knowledge of any of them: solving linear equations, solving quadratic equations, and calculating probabilities. This means that their probability of answering a question correctly for each skill is 0.5.

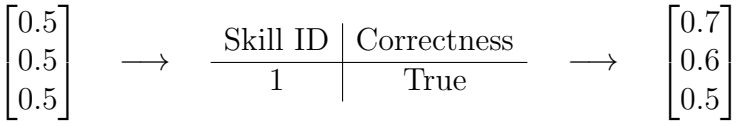

Figure 4: An example of knowledge update after answering a question correctly.

If, in this state, they correctly answer a question about solving linear equations, we can assume they have gained some related knowledge. As a result, their probability of correctly answering another question on linear equations should increase.

Since solving quadratic equations is related to linear equations, this probability may also increase.

However, calculating probabilities is an unrelated skill, so answering a linear equation correctly provides no indication of improvement in probability calculations. Thus, the probability of answering a probability-related question correctly remains unchanged.

Note that this is an over-simplified scenario—it is also possible that the person simply guessed the correct answer without actually acquiring new knowledge.

4.2 Loss Calculation

The loss is calculated based on how well the predicted probabilities align with the actual answers to the subsequent questions. Specifically, after obtaining the output probabilities at a given item in a sequence, we look at the next item's skill IDs and correctness. Using the skill IDs, we extract the corresponding probabilities from the knowledge vector and compute the Binary Cross-Entropy (BCE) loss (or AUC) between these probabilities and the correctness of the answers.

This also implies that for a sequence of length n, the inputs consist of the first $n-1$ items, while the outputs correspond to the last $n-1$ items.

Formal Definition

Let X be an input sequence of dimensions $n \times d$, where each row represents an item in the sequence:

- The first column contains the skill IDs (a set containing at least one integer from 1 to m, where m is the total number of skills - at least one, as explained in Subsection 2.2).
- The second column contains the correctness values (0 or 1).
- Additional columns may exist if further features are available, otherwise $d = 2$.
- $X^{\text{input}} \coloneqq X$ without the last row (dimensions: $(n-1) \times d$).
- $X^{\text{output}} \coloneqq X$ without the first row (dimensions: $(n-1) \times d$).

The network produces predictions for X^{input} denoted by Y, a matrix of dimensions $(n-1) \times m$.

The loss function is then computed as:

$$\text{Loss}(X, Y) = \text{BCE}\left(X_{:,2}^{\text{output}},\ Y[X_{:,1}^{\text{output}}]\right)$$

where the extracted values from Y are computed as:

$$Y[X_{:,1}^{\text{output}}]_i = \frac{1}{|X_{i,1}^{\text{output}}|} \sum_{j \in X_{i,1}^{\text{output}}} Y_{i,j}$$

That is, for each sequence step i, the corresponding predicted probability is the average of the values $Y_{i,j}$ over all indices j present in $X_{i,1}^{\text{output}}$.

4.3 Architecture

The architecture of my model is largely based on the original design, with modifications in the embedding creation and using different hyperparameters.

Instead of embedding a `(correct, skill ID)` pair, my approach treats correctness and skill ID separately. As discussed in Subsection 2.2, a question may belong to multiple skills. To handle this, each skill ID is mapped to its corresponding **embedding**, and their **mean** is computed to form a single representation for the question.

This aggregated embedding is then **concatenated** with the correctness value and any additional features (if present). The resulting vector is passed to an **LSTM layer**, followed by a **dropout layer** to prevent overfitting. Finally, a **sigmoid activation function** is applied to obtain the predicted probabilities.

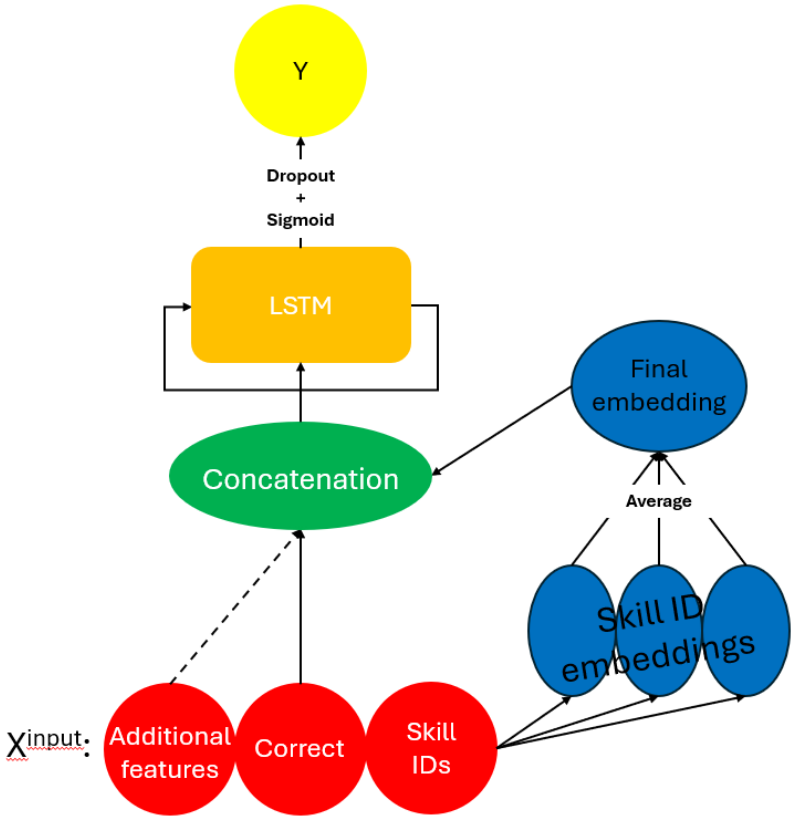

Figure 5: Visualization of the architecture

4.4 Additional Features

From the dataset, I extracted and cleaned multiple features that contributed to performance improvements, however not significantly:

- **Ease:** The ratio of correct answers to total attempts for a given question, indicating its relative difficulty.
- **Bottom hint used:** A binary indicator of whether the student used the final hint before submitting their answer.
- **Response time (ms):** The time, in milliseconds, between when the question appeared and when the student submitted their response.

4.5 Modeling Results

During the modeling phase, I again split the dataset into two main sections: **80% training** set and **20% test** set.

As the original paper did not specify all hyperparameters (e.g., dropout factor), I performed an extensive hyperparameter tuning process. Using **5-fold cross-validation** on the training set, I selected the hyperparameters that resulted in the highest **average AUC** across the folds.

The implementation of this process, along with the final selected hyperparameters, is available in my repository [3]. The achieved AUC scores are as follows:

Model	Train Set	Test Set
Simple	0.799	0.794
Complex	0.816	0.802

Table 2

To ensure that my model does not suffer from overfitting, I also report AUC values on the training set.

The **Simple** model refers to the version without additional features, and the **Complex** model includes the features described in Subsection 4.4. Notably, adding three additional features led to only a **marginal AUC improvement** of less than **0.01**, suggesting that **skill ID** and **correctness** contain most of the predictive information.

Despite employing a more sophisticated embedding method and comprehensive hyperparameter tuning, my results fall short of the **0.86 AUC** reported in the original paper. This discrepancy can be because of the data duplication issue discussed earlier, which likely inflated the performance in their results.

4.6 Knowledge Structure

My final contribution was to investigate whether a meaningful structure between different skills could be inferred from their trained embeddings.

The motivation behind this analysis comes from the original paper, which demonstrated the feasibility of identifying skill relationships on two other datasets using a different method. Specifically, they computed influence scores between skills—measuring

how correctly answering a question in skill i increases the probability of correctly answering questions in other skills. If this probability shift exceeds a predefined threshold, a connection between the skills is established.

Surprisingly, the authors did not discuss the knowledge structure of the Assistments dataset, despite having access to the names of most of the skills. After implementing their influence scoring method, I found that, in this dataset, the approach fails—it exhibits random influences rather than meaningful connections.

Knowledge Structure via Embeddings

Given the limitations of the influence score method, I explored an alternative approach: analyzing skill relationships through their trained embeddings. I examined the individual embeddings of all skills using multiple techniques:

- t-SNE (t-distributed Stochastic Neighbor Embedding): Visualizing the embeddings in 2D and 3D.
- Hierarchical and K-means Clustering.

Unfortunately, the results revealed no meaningful structure—only a few skill pairs were positioned close together in a way that seemed reasonable, but given the large number of skills (109), this proximity could be attributed to randomness.

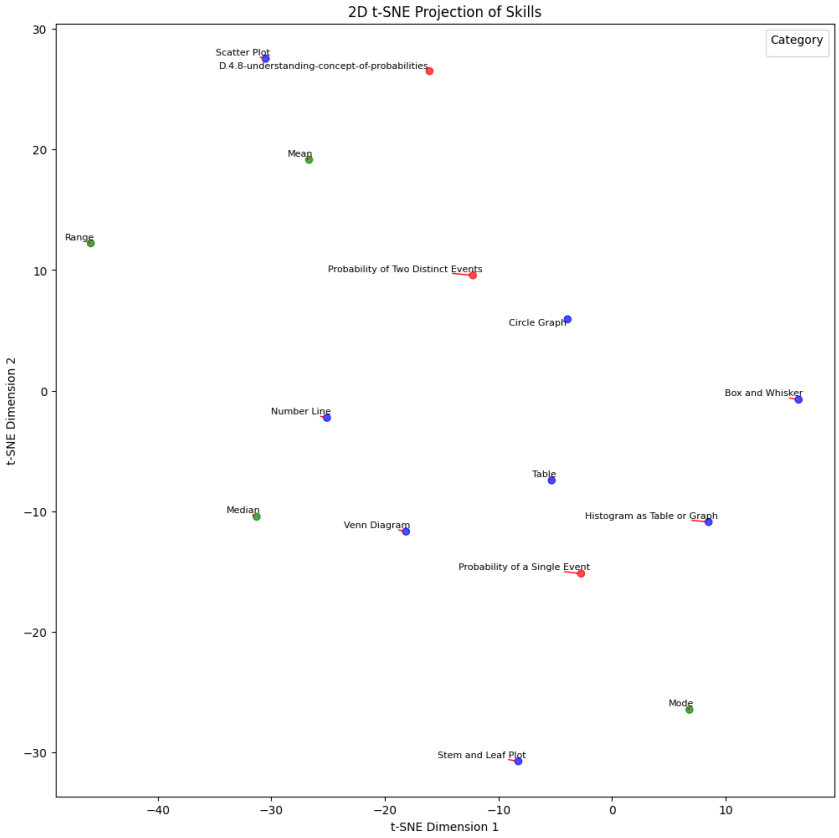

Figure 6

I assumed that having too many skills might be overwhelming for the model. To test this, I filtered the dataset to include only 15 distinct skills that clearly belonged to three

mathematical clusters. I then trained a new model and reassessed the structure using t-SNE and clustering. One corresponding visualization is presented in Figure 6, where each colour represents a knowledge cluster, which I have labeled.

Despite reducing the skill set, the results **remained random**, suggesting that this approach might not be effective for uncovering knowledge structures. Consequently, I decided to check the method on a synthetic dataset.

Embeddings in a Synthetic Dataset

To check if this approach is even able to detect skill relationships, I simulated a synthetic dataset where each student starts with a random probability for each skill. When presented with a question related to a particular skill, the student answers correctly based on the corresponding probability. Upon a correct answer, the skill probability increases by a fixed value, as illustrated in Figure 7. If the answer is incorrect, the probability is reduced: it is divided by 3 and then subtracted.

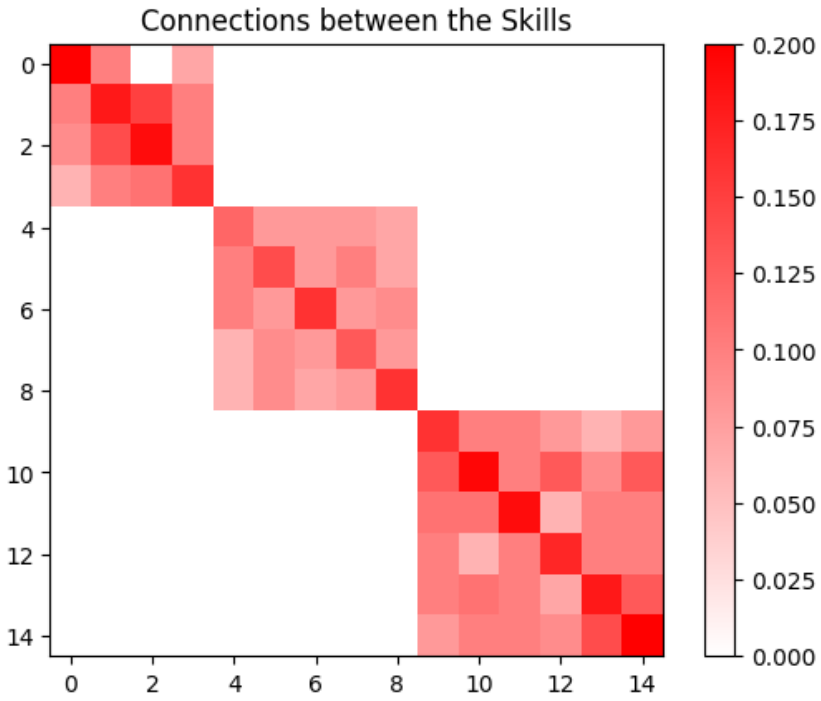

Figure 7: Visualization of the simulated skill probability adjustment matrix.

This setup clearly defines three distinct skill groups (1, 2, and 3), which influence each other. I then trained a DKT RNN on this synthetic dataset, achieving a relatively high AUC score of 0.83.

What was particularly important to observe was that the feature embeddings were **able to capture** the skill relationships quite well, with the exception of one skill (1.3), which was not properly recognized in terms of its connection to the other skills.

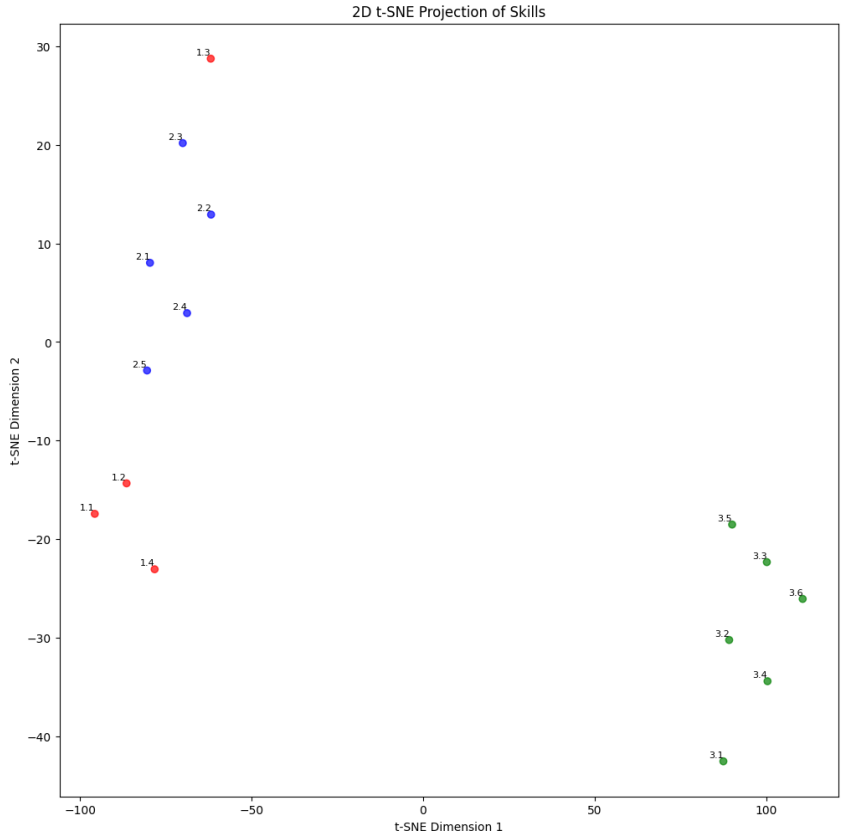

Figure 8: 2D visualization of skill embeddings in the synthetic dataset.

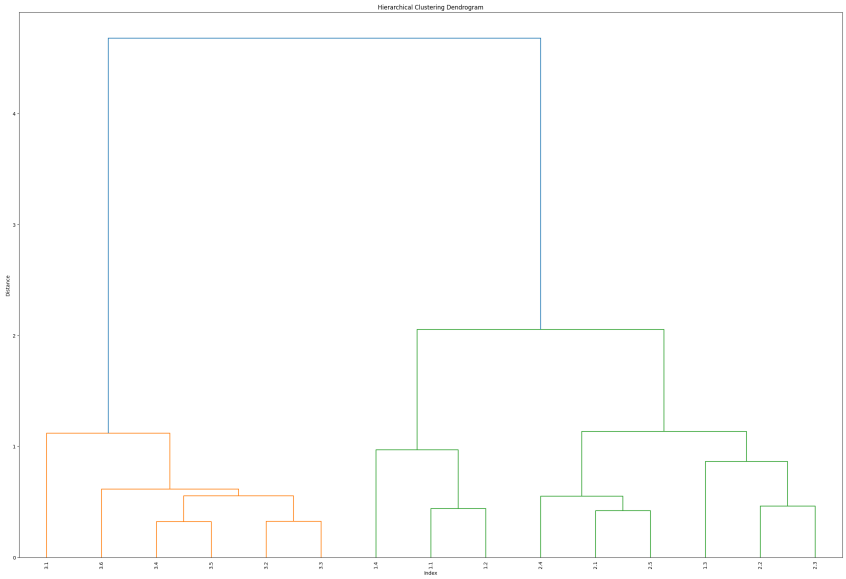

Figure 9: Hierarchical clustering of skill embeddings in the synthetic dataset.

5 Future Work

Since the release of the original paper, numerous advanced architectures for sequential data have been proposed. For example, the Transformer architecture—applied to knowledge tracing in 2020 [7]—has demonstrated significant improvements. Future work could try out the xLSTM [6] architecture to potentially enhance model performance and reveal latent structures.

Additionally, the application of explainability techniques, like SHAP values, could offer insights into the relative importance of features such as skill IDs and correctness in the model's predictions.

Experimenting with the proposed structure detection method via embeddings on alternative datasets could provide a deeper understanding of its applicability and effectiveness.

6 Conclusion

In this project, I revisited knowledge tracing by implementing and extending both the Bayesian Knowledge Tracing (BKT) and Deep Knowledge Tracing (DKT) models.

Through extensive data cleaning, a custom multi-skill representation, and careful hyperparameter tuning, competitive AUC scores were achieved.

Although the enhanced embedding methods and architectural modifications worked well in realizing the latent structure of synthetic datasets, the results on real-world data did not meet our expectations.

These findings highlight the challenges in accurately modeling student learning and emphasize the need for further experimentation.

References

[1] PIECH, C., BAKER, R. S. J., BERNSTEIN, D. J., CESCOVINI, J., and WONG, S. Deep knowledge tracing. In *Proceedings of the 8th International Conference on Educational Data Mining*, pp. 453–460, 2015
https://stanford.edu/~cpiech/bio/papers/deepKnowledgeTracing.pdf

[2] Massachusetts Institute of Technology and Worcester Polytechnic Institute ASSISTments 2009-10 Dataset
https://sites.google.com/site/assistmentsdata/an-explanation-on-how-to-interpret-our-data-sets

[3] https://github.com/botond0401/personalized-education

[4] GUO, C. and BERKHAHN, F. Entity Embeddings of Categorical Variables., 2016
https://arxiv.org/abs/1604.06737

[5] CORBETT, A. T., AND ANDERSON, J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278, 1994.
https://doi.org/10.1007/BF01099821

[6] BECK, M., PÖPPEL, K., SPANRING, M., AUER, A., PRUDNIKOVA, O., KOPP, M., KLAMBAUER, G., BRANDSTETTER, J., AND HOCHREITER, S. xLSTM: Extended Long Short-Term Memory. *arXiv preprint arXiv:2405.04517*, 2024.
https://arxiv.org/abs/2405.04517

[7] PU, S., YUDELSON, M., OU, L., AND HUANG, Y. Deep Knowledge Tracing with Transformers. In: Bittencourt, I., Cukurova, M., Muldner, K., Luckin, R., Millán, E. (eds) Artificial Intelligence in Education. AIED 2020. Lecture Notes in Computer Science vol. 12164, pp. 252–256, 2020.
https://doi.org/10.1007/978-3-030-52240-7_46